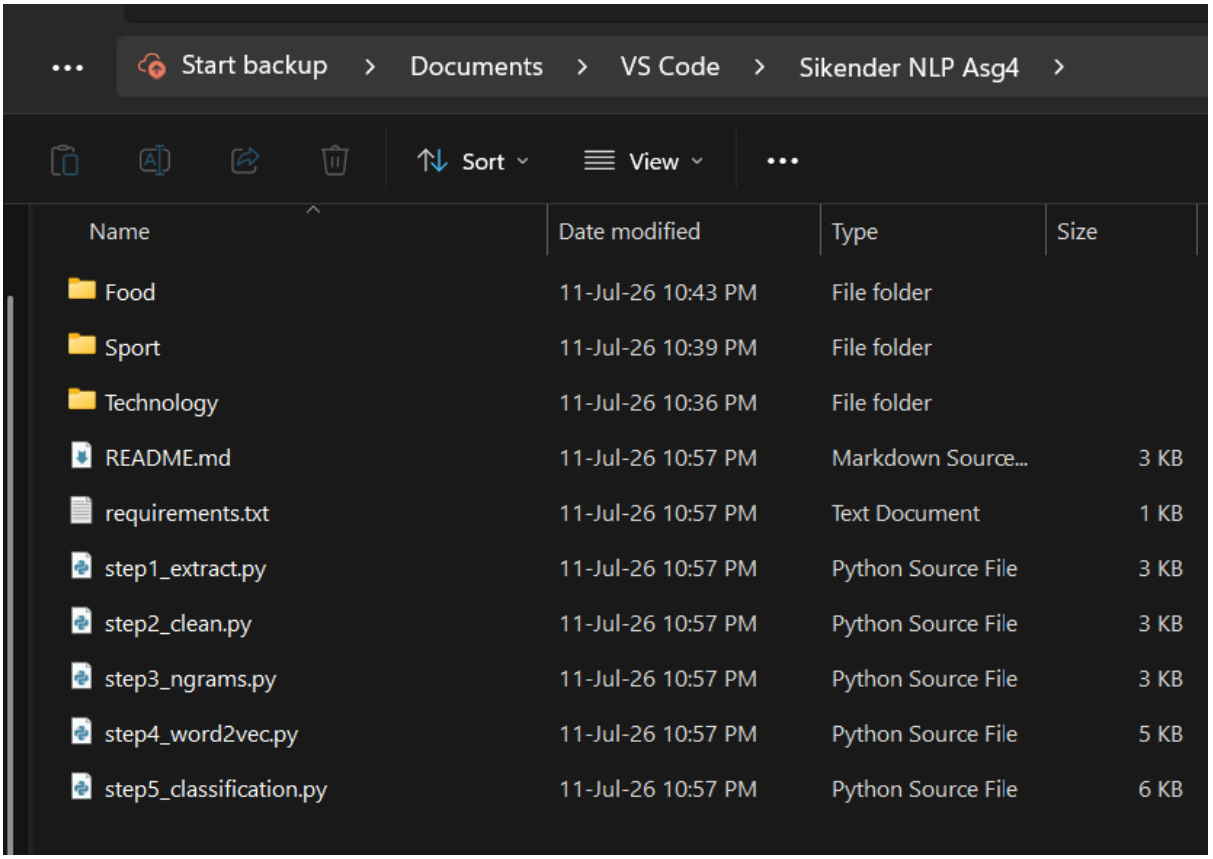
Assignment 04: PDF-Based NLP Pipeline

Sikender Gillani BSAI-23F-0062

Downloaded some pdfs and renamed them into 3 separate folders.



STEP 1: PDF Text Extraction and CSV Dataset Creation

Reads all PDFs from category-wise folders and builds a structured CSV with columns: file_name, category, raw_text

Code:

step1_extract.py > build_dataset

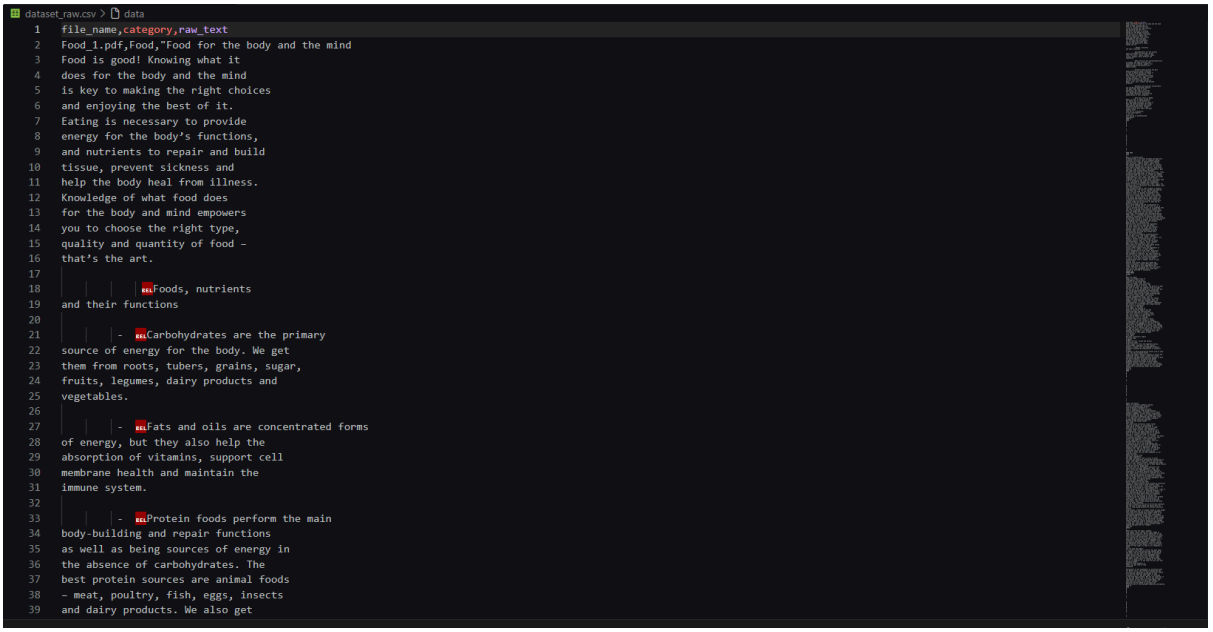
```python
import os
import re
import csv
import fitz  # PyMuPDF


CORPUS_DIR = "."  # <-- point this to the folder containing your category-wise PDF folders
OUTPUT_CSV = "dataset_raw.csv"


def extract_text_from_pdf(pdf_path):
    """Extract raw text from a PDF file using PyMuPDF (fitz)."""
    text = ""
    try:
        doc = fitz.open(pdf_path)
        for page in doc:
            text += page.get_text()
        doc.close()
    except Exception as e:
        print(f"  [WARNING] Failed to read {pdf_path}: {e}")
    return text


def build_dataset(corpus_dir):
    rows = []
    categories = [d for d in os.listdir(corpus_dir)
                  if os.path.isdir(os.path.join(corpus_dir, d))]

    print(f"Found categories: {categories}")

    for category in categories:
        folder_path = os.path.join(corpus_dir, category)
        pdf_files = [f for f in os.listdir(folder_path) if f.lower().endswith(".pdf")]
        print(f"  {category}: {len(pdf_files)} PDF file(s)")

        for file_name in pdf_files:
            pdf_path = os.path.join(folder_path, file_name)
            raw_text = extract_text_from_pdf(pdf_path)
            raw_text = raw_text.strip()
```

step1_extract.py > build_dataset

```python
def build_dataset(corpus_dir):
            raw_text = raw_text.strip()

            if len(raw_text) < 10:
                print(f"  [SKIPPED] {file_name} - little/no extractable text (likely scanned image)")
                continue

            rows.append({
                "file_name": file_name,
                "category": category,
                "raw_text": raw_text
            })

    return rows


if __name__ == "__main__":
    rows = build_dataset(CORPUS_DIR)

    with open(OUTPUT_CSV, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=["file_name", "category", "raw_text"])
        writer.writeheader()
        writer.writerows(rows)

    print(f"\nSaved {len(rows)} documents to {OUTPUT_CSV}")
```

Output:

```
PS C:\Users\Hp\Documents\VS Code\Sikender NLP Asg4> & C:\Users\Hp\AppData\Lo
Found categories: ['Food', 'Sport', 'Technology']
  Food: 3 PDF file(s)
  [SKIPPED] Food_2.pdf - little/no extractable text (likely scanned image)
  Sport: 3 PDF file(s)
  Technology: 6 PDF file(s)

Saved 11 documents to dataset_raw.csv
PS C:\Users\Hp\Documents\VS Code\Sikender NLP Asg4>
```



```
dataset_raw.csv > data
1   file_name,category,raw_text
2   Food_1.pdf,Food,"Food for the body and the mind
3   Food is good! Knowing what it
4   does for the body and the mind
5   is key to making the right choices
6   and enjoying the best of it.
7   Eating is necessary to provide
8   energy for the body's functions,
9   and nutrients to repair and build
10  tissue, prevent sickness and
11  help the body heal from illness.
12  Knowledge of what food does
13  for the body and mind empowers
14  you to choose the right type,
15  quality and quantity of food –
16  that's the art.
17
18          Foods, nutrients
19  and their functions
20
21          - Carbohydrates are the primary
22  source of energy for the body. We get
23  them from roots, tubers, grains, sugar,
24  fruits, legumes, dairy products and
25  vegetables.
26
27          - Fats and oils are concentrated forms
28  of energy, but they also help the
29  absorption of vitamins, support cell
30  membrane health and maintain the
31  immune system.
32
33          - Protein foods perform the main
34  body-building and repair functions
35  as well as being sources of energy in
36  the absence of carbohydrates. The
37  best protein sources are animal foods
38  – meat, poultry, fish, eggs, insects
39  and dairy products. We also get
```

STEP 2: Text Cleaning and Preprocessing

Cleaning steps: lowercase, URL removal, email removal, number removal, punctuation removal, extra-space normalization, stop-word removal, tokenization, removal of very short tokens.

Code:

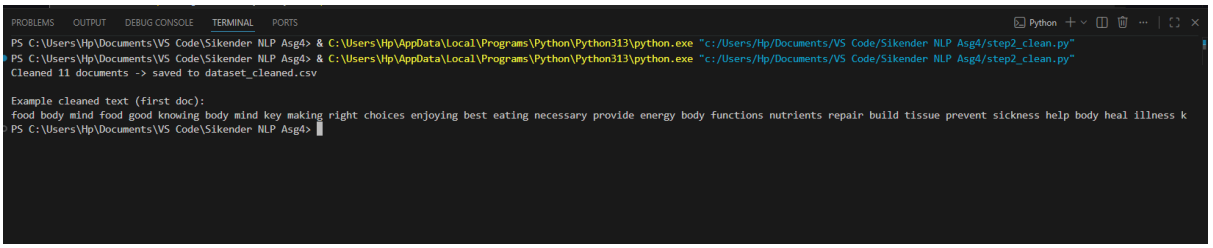
step2_clean.py > clean_text

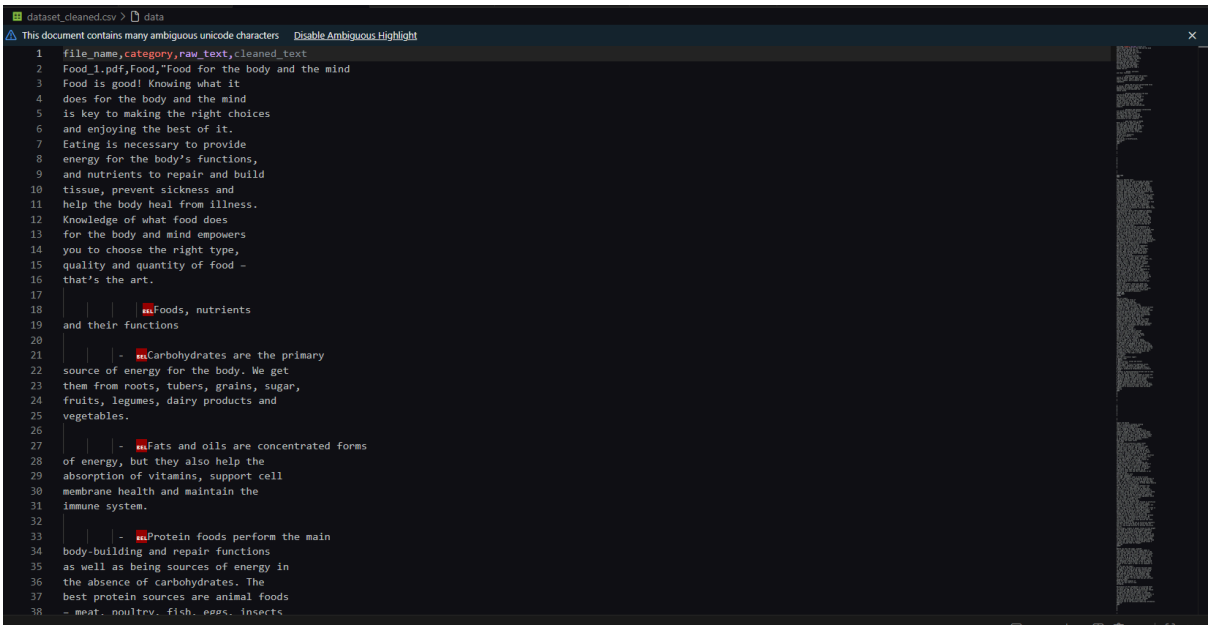
```python
import re
import csv
import sys
import nltk
from nltk.corpus import stopwords
csv.field_size_limit(sys.maxsize)

nltk.download("stopwords", quiet=True)
nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)

INPUT_CSV = "dataset_raw.csv"
OUTPUT_CSV = "dataset_cleaned.csv"

STOP_WORDS = set(stopwords.words("english"))


def clean_text(text):
    """Full cleaning pipeline using regular expressions."""
    text = text.lower()                                  # lowercase
    text = re.sub(r"http\S+|www\.\S+", " ", text)        # remove URLs
    text = re.sub(r"\S+@\S+\.\S+", " ", text)            # remove emails
    text = re.sub(r"\d+", " ", text)                     # remove numbers
    text = re.sub(r"[^\w\s]", " ", text)                 # remove punctuation
    text = re.sub(r"\s+", " ", text).strip()             # normalize spaces

    tokens = text.split()                                # tokenization
    tokens = [t for t in tokens if t not in STOP_WORDS]  # stop-word removal
    tokens = [t for t in tokens if len(t) > 2]           # remove very short tokens

    return tokens


if __name__ == "__main__":
    rows = []
    with open(INPUT_CSV, "r", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            tokens = clean_text(row["raw_text"])
```

step2_clean.py > ...

```python
            rows.append(row)

    with open(OUTPUT_CSV, "w", newline="", encoding="utf-8") as f:
        fieldnames = ["file_name", "category", "raw_text", "cleaned_text"]
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)

    print(f"Cleaned {len(rows)} documents -> saved to {OUTPUT_CSV}")
    print("\nExample cleaned text (first doc):")
    print(rows[0]["cleaned_text"][:200])
```

Output:





STEP 3: N-gram Generation and Frequency Graphs

Generates unigram, bigram, and trigram versions of the cleaned corpus and plots top-word / n-gram frequency graphs.

Code:

step3_ngrams.py > plot_top_freq

```python
import csv
import sys
from collections import Counter
import matplotlib.pyplot as plt
csv.field_size_limit(sys.maxsize)

INPUT_CSV = "dataset_cleaned.csv"


def get_ngrams(tokens, n):
    return ["_".join(tokens[i:i + n]) for i in range(len(tokens) - n + 1)]


def plot_top_freq(counter, title, filename, top_n=15):
    common = counter.most_common(top_n)
    labels = [c[0] for c in common]
    values = [c[1] for c in common]

    plt.figure(figsize=(10, 6))
    plt.barh(labels[::-1], values[::-1], color="steelblue")
    plt.title(title)
    plt.xlabel("Frequency")
    plt.tight_layout()
    plt.savefig(filename, dpi=120)
    plt.close()
    print(f"Saved graph: {filename}")


if __name__ == "__main__":
    rows = []
    with open(INPUT_CSV, "r", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            rows.append(row)

```

step3_ngrams.py > plot_top_freq

```python
    all_tokens = []
    unigram_docs, bigram_docs, trigram_docs = [], [], []

    for row in rows:
        tokens = row["cleaned_text"].split()
        all_tokens.extend(tokens)

        unigrams = get_ngrams(tokens, 1)
        bigrams = get_ngrams(tokens, 2)
        trigrams = get_ngrams(tokens, 3)

        unigram_docs.append(" ".join(unigrams))
        bigram_docs.append(" ".join(bigrams))
        trigram_docs.append(" ".join(trigrams))

    # Save n-gram versions of the corpus
    with open("dataset_ngrams.csv", "w", newline="", encoding="utf-8") as f:
        fieldnames = ["file_name", "category", "unigrams", "bigrams", "trigrams"]
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        for row, u, b, t in zip(rows, unigram_docs, bigram_docs, trigram_docs):
            writer.writerow({
                "file_name": row["file_name"],
                "category": row["category"],
                "unigrams": u,
                "bigrams": b,
                "trigrams": t
            })
    print("Saved dataset_ngrams.csv")

    # Frequency counters
    unigram_counter = Counter(all_tokens)
    bigram_counter = Counter(" ".join(bigram_docs).split())
    trigram_counter = Counter(" ".join(trigram_docs).split())

    # Required graphs
    plot_top_freq(unigram_counter, "Top-Word Frequency (After Preprocessing)", "graph_top_words.png")
    plot_top_freq(unigram_counter, "Unigram Frequency", "graph_unigram_freq.png")
    plot_top_freq(bigram_counter, "Bigram Frequency", "graph_bigram_freq.png")
    plot_top_freq(trigram_counter, "Trigram Frequency", "graph_trigram_freq.png")

    print("\nTop 10 unigrams:", unigram_counter.most_common(10))
```

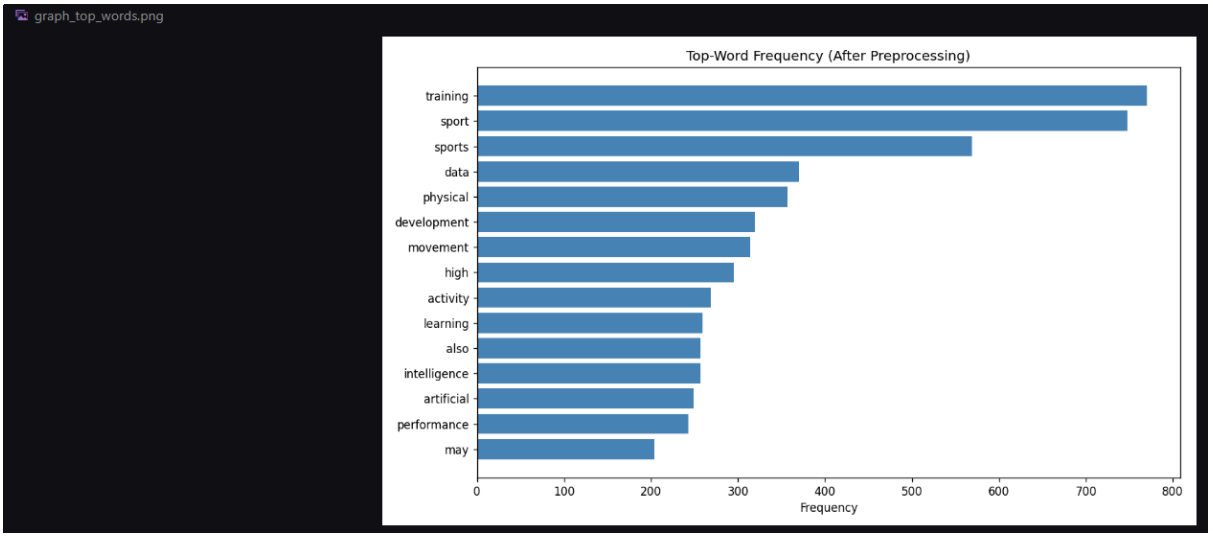
Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                                    Python

PS C:\Users\Hp\Documents\VS Code\Sikender NLP Asg4> & C:\Users\Hp\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Hp/Documents/VS Code/Sikender NLP Asg4/step2_clean.py"
PS C:\Users\Hp\Documents\VS Code\Sikender NLP Asg4> & C:\Users\Hp\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Hp/Documents/VS Code/Sikender NLP Asg4/step3_ngrams.py"
Saved dataset_ngrams.csv
Saved graph: graph_top_words.png
Saved graph: graph_unigram_freq.png
Saved graph: graph_bigram_freq.png
Saved graph: graph_trigram_freq.png

Top 10 unigrams: [('training', 770), ('sport', 748), ('sports', 569), ('data', 370), ('physical', 357), ('development', 319), ('movement', 314), ('high', 295), ('activity', 269), ('learning', 259)]
PS C:\Users\Hp\Documents\VS Code\Sikender NLP Asg4>
```

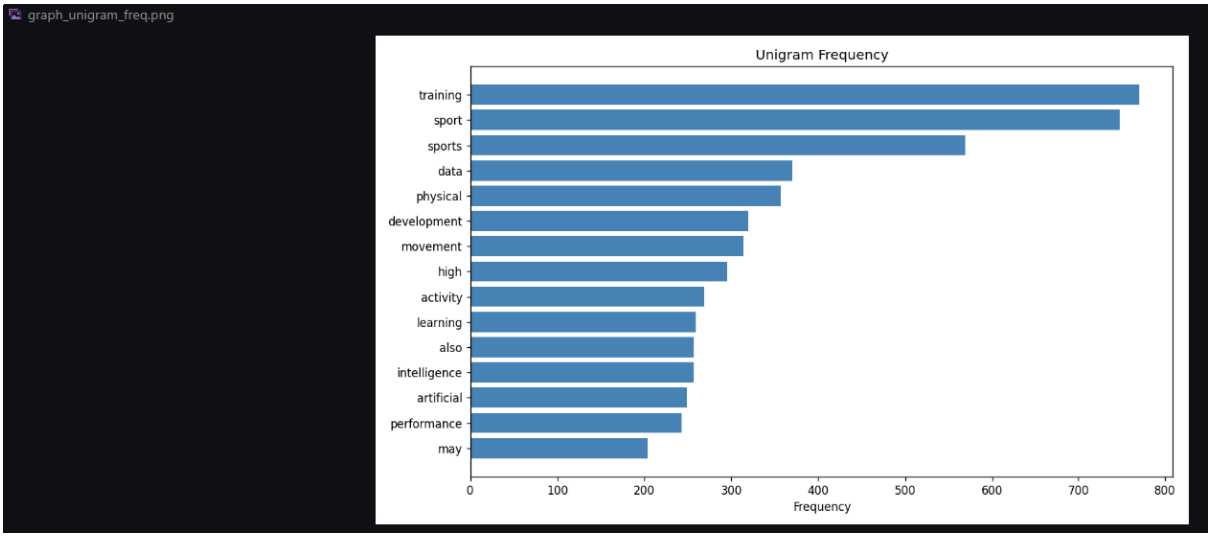
dataset_ngrams.csv > data

⚠ This document contains many ambiguous unicode characters Disable Ambiguous Highlight

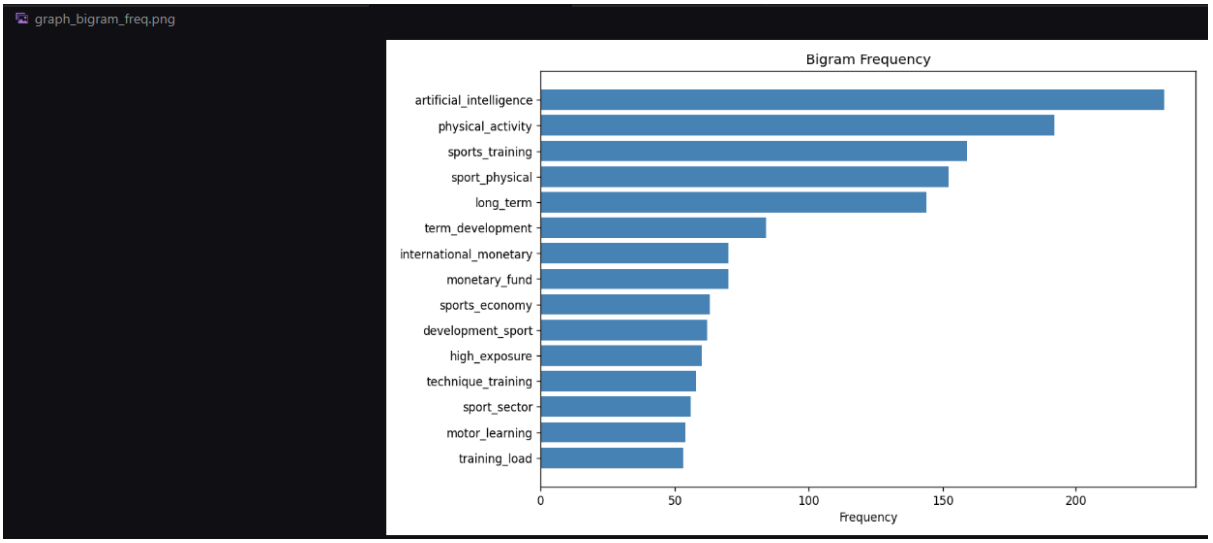
```
1   file_name,category,unigrams,bigrams,trigrams
2   Food_1.pdf,Food,food body mind food good knowing body mind key making right choices enjoying best eating necessary provide energy body functions nutrients repair build tissue
3   Food_3.pdf,Food,today dietitian protein content foods meat poultry eggs food cooked serving size calories protein chicken skinless steak turkey roasted lamb pork ham egg larg
4   sport_1.pdf,Sport,sports people planet collaboration oliver wyman images adobe stock getty images disclaimer document published world economic forum contribution project insi
5   sport_2.pdf,Sport,sports training definitions aims functions characteristics general word training commonly used term human language broad sense training may define organisec
6   sport_3.pdf,Sport,long term development sport physical activity writers colin higgs phd richard way mba vicki harber phd paul jurbala phd istvan balyi editors contributors re
7   tech_01.pdf,Technology,edited share members september artificial intelligence sport sector paper share initiative european commission directorate general education youth spor
8   tech_02.pdf,Technology,journal social horizons vol september jsh doi original article corresponding author hina munir assistant professor department education women universit
9   tech_03.pdf,Technology,introduction artificial intelligence fundamental concepts melike palsü kurt learn artificial intelligence machines think think turing test chinese room
10  tech_04.pdf,Technology,october money capital markets information technology departments powering digital economy opportunities risks artificial intelligence finance prepared
11  tech_05.pdf,Technology,imf staff discussion notes sdns showcase policy related analysis research developed imf staff members published elicit comments encourage debate views
12  tech_06.pdf,Technology,accumulate technology celebrates anniversary optimistic micro led future dis play mainstream technology login registration track fan group author techr
13
```



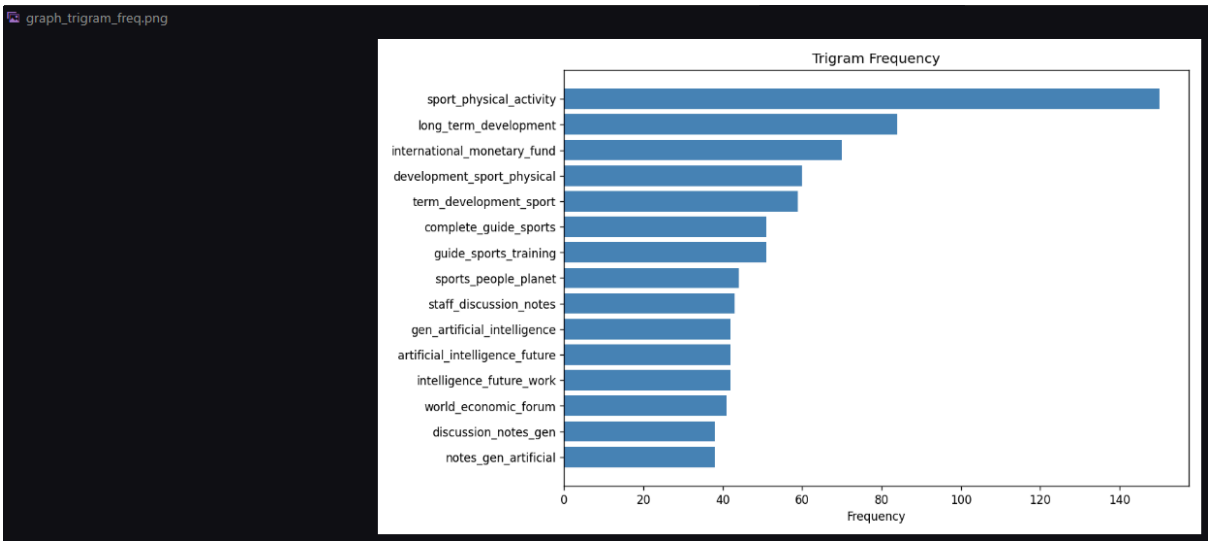
After cleaning the text by removing stopwords, punctuation, and numbers, training and sport came out as the most frequent words, appearing over 700 times each. This makes sense since my Technology folder had 6 PDFs, double the other two categories, and several of those tech PDFs were actually about AI and sports training topics rather than pure tech news, which is why sport related words dominate even the overall corpus.



This graph is essentially the single word view of the corpus. The heavy presence of training, sport, and sports at the top, followed by data, physical, development, and movement, shows the corpus content leans toward sports science and physical training topics, likely because some of my Technology documents discussed AI in the context of sports performance data.



The bigram graph shows two word phrases instead of isolated words. Artificial intelligence is by far the most common bigram with over 200 occurrences, which tells me my Technology PDFs are heavily focused on AI specifically. Right behind it, physical activity and sports training confirm the Sport category's core theme. Interestingly, international monetary and monetary fund also show up. This comes from one of my tech PDFs, an IMF staff discussion note, which pulled in some economics vocabulary.



Trigrams are naturally rarer than bigrams since exact three word sequences repeat less often across only 11 documents. Sport physical activity is the clear leader with about 150 occurrences, confirming the Sport documents use this exact phrase repeatedly. I also see the international monetary fund and world economic forum. These come from one specific Technology PDF, an IMF discussion paper, showing how a single document can dominate trigram counts when the corpus is this small.

STEP 4 Part a: Next Word Prediction using Word2Vec with CBOW & Skip-gram

Trains CBOW and Skip-gram models on unigram, bigram, and trigram versions of the cleaned corpus, and compares predictions.

Code:

step4_word2vec.py > train_models

```python
import csv
import sys
import matplotlib.pyplot as plt
from gensim.models import Word2Vec

csv.field_size_limit(sys.maxsize)

INPUT_CSV = "dataset_cleaned.csv"


def train_models(sentences, label):
    """Train CBOW (sg=0) and Skip-gram (sg=1) on a list of tokenized sentences."""
    cbow = Word2Vec(sentences, vector_size=100, window=3, min_count=1, sg=0, epochs=100)
    skipgram = Word2Vec(sentences, vector_size=100, window=3, min_count=1, sg=1, epochs=100)
    print(f"Trained CBOW and Skip-gram on {label} corpus "
          f"(vocab size: {len(cbow.wv)})")
    return cbow, skipgram


def safe_most_similar(model, word, topn=5):
    if word in model.wv:
        return model.wv.most_similar(word, topn=topn)
    return []


def compare_and_plot(cbow, skipgram, test_words, ngram_label):
    """Print comparison table and plot similarity scores for one test word."""
    print(f"\n--- CBOW vs Skip-gram comparison ({ngram_label}) ---")
    for word in test_words:
        cbow_sim = safe_most_similar(cbow, word)
        sg_sim = safe_most_similar(skipgram, word)
        print(f"\nInput word: '{word}'")
        print(f"  CBOW predictions     : {cbow_sim}")
        print(f"  Skip-gram predictions: {sg_sim}")

    # Plot for the first valid test word
    for word in test_words:
        cbow_sim = safe_most_similar(cbow, word)
        sg_sim = safe_most_similar(skipgram, word)
        if cbow_sim and sg_sim:
```

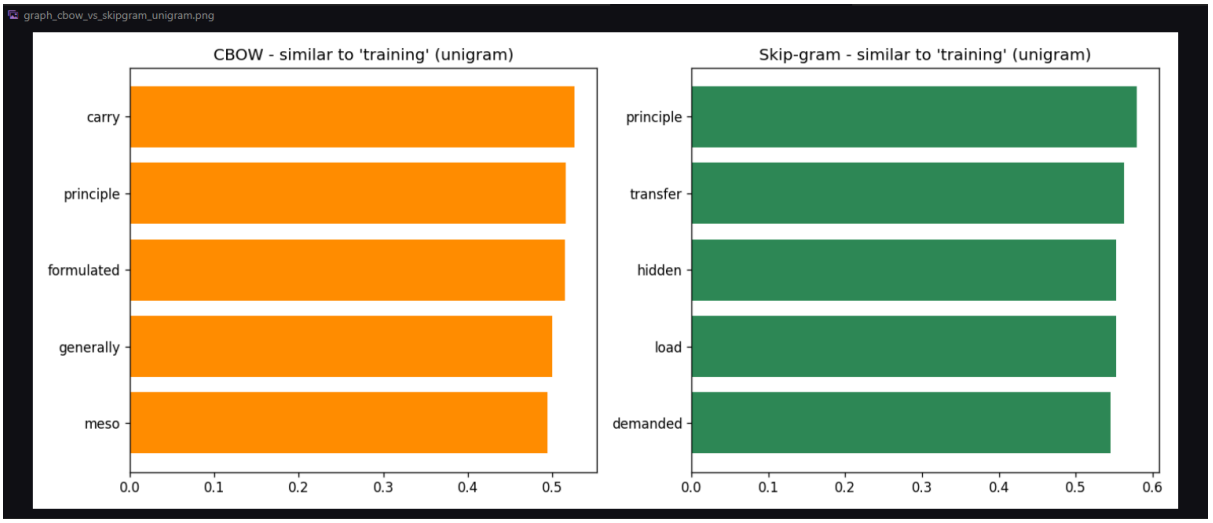
step4_word2vec.py > train_models

```
26  def compare_and_plot(cbow, skipgram, test_words, ngram_label):
41              words_c = [w for w, s in cbow_sim]
42              scores_c = [s for w, s in cbow_sim]
43              words_s = [w for w, s in sg_sim]
44              scores_s = [s for w, s in sg_sim]
45
46              fig, axes = plt.subplots(1, 2, figsize=(12, 5))
47              axes[0].barh(words_c[::-1], scores_c[::-1], color="darkorange")
48              axes[0].set_title(f"CBOW - similar to '{word}' ({ngram_label})")
49              axes[1].barh(words_s[::-1], scores_s[::-1], color="seagreen")
50              axes[1].set_title(f"Skip-gram - similar to '{word}' ({ngram_label})")
51              plt.tight_layout()
52              fname = f"graph_cbow_vs_skipgram_{ngram_label}.png"
53              plt.savefig(fname, dpi=120)
54              plt.close()
55              print(f"Saved graph: {fname}")
56              break
57
58
59  if __name__ == "__main__":
60      rows = []
61      with open(INPUT_CSV, "r", encoding="utf-8") as f:
62          reader = csv.DictReader(f)
63          for row in reader:
64              rows.append(row)
65
66      unigram_sentences = [row["cleaned_text"].split() for row in rows]
67
68      def make_ngram_sentences(n):
69          sentences = []
70          for row in rows:
71              tokens = row["cleaned_text"].split()
72              grams = ["_".join(tokens[i:i + n]) for i in range(len(tokens) - n + 1)]
73              if grams:
74                  sentences.append(grams)
75          return sentences
76
77      bigram_sentences = make_ngram_sentences(2)
78      trigram_sentences = make_ngram_sentences(3)
79
```

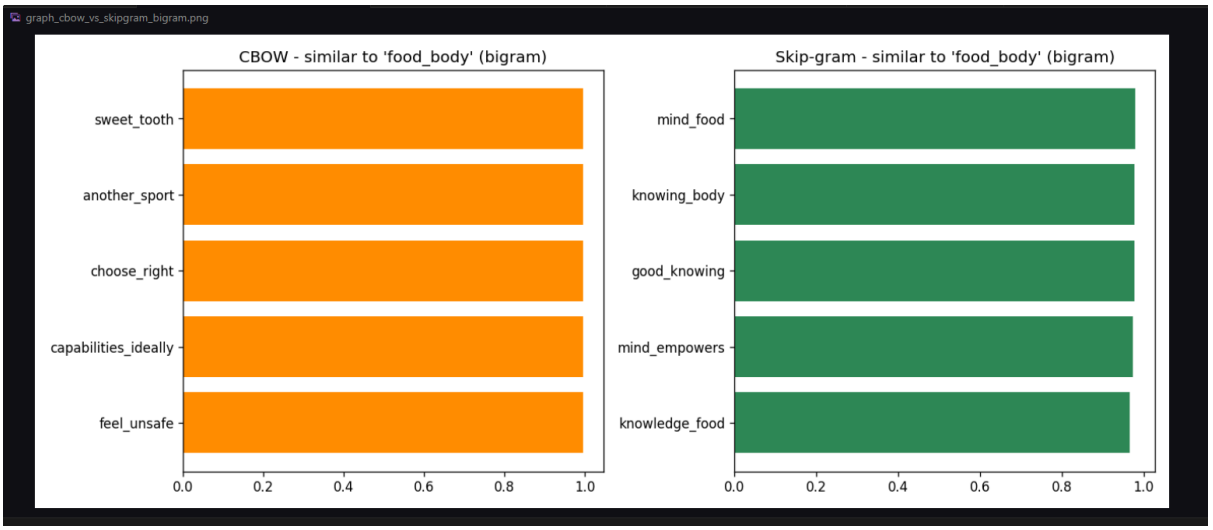
```python
    # Pick a few frequent words from the corpus itself as test examples and change these based on your actual corpus vocabulary
    all_words = [w for sent in unigram_sentences for w in sent]
    from collections import Counter
    common_words = [w for w, c in Counter(all_words).most_common(10)]
    test_words = common_words[:3]
    print("Using test words from corpus:", test_words)

    # ---- Unigram models ----
    cbow_u, sg_u = train_models(unigram_sentences, "unigram")
    compare_and_plot(cbow_u, sg_u, test_words, "unigram")

    # ---- Bigram models ----
    cbow_b, sg_b = train_models(bigram_sentences, "bigram")
    bigram_test = [s.split()[0] for s in [bigram_sentences[0][0]] if bigram_sentences]
    compare_and_plot(cbow_b, sg_b, bigram_test, "bigram")

    # ---- Trigram models ----
    cbow_t, sg_t = train_models(trigram_sentences, "trigram")
    trigram_test = [trigram_sentences[0][0]] if trigram_sentences else []
    compare_and_plot(cbow_t, sg_t, trigram_test, "trigram")

    print("\nDone. Models trained and comparison graphs saved.")
```

Output:

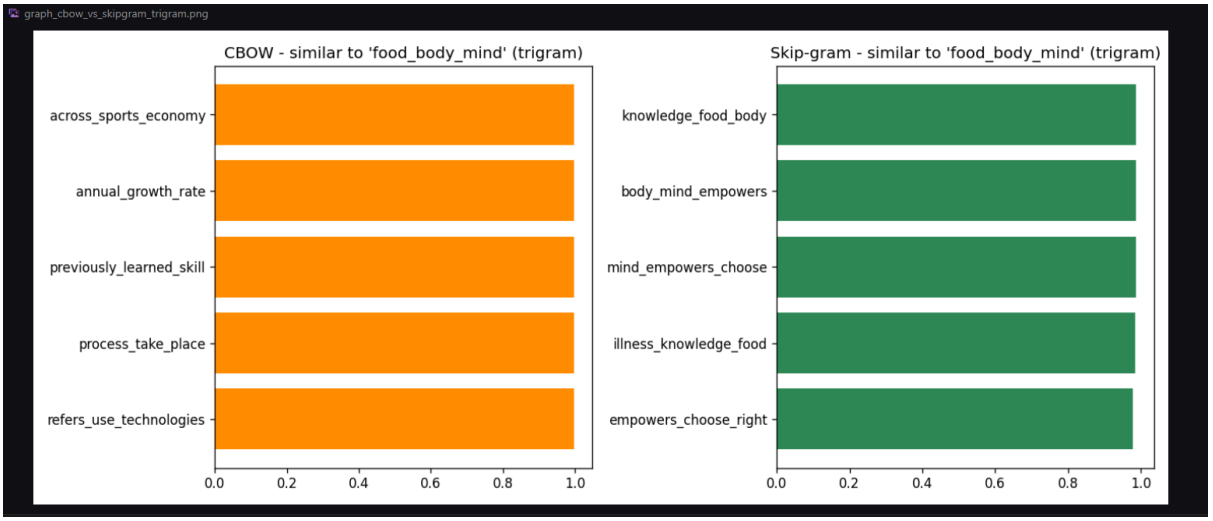
```
PS C:\Users\Hp\Documents\VS Code\Sikender NLP Asg4> & C:\Users\Hp\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Hp/Documents/VS Code/Sikender NLP Asg4/step4_word2vec.py"
Using test words from corpus: ['training', 'sport', 'sports']
Trained CBOW and Skip-gram on unigram corpus (vocab size: 8579)

--- CBOW vs Skip-gram comparison (unigram) ---

Input word: 'training'
  CBOW predictions     : [('carry', 0.5262035727500916), ('principle', 0.5156347155570984), ('formulated', 0.5144956707954407), ('generally', 0.5000763535499573), ('meso', 0.4939110577106476
)]
  Skip-gram predictions: [('principle', 0.5795835256576538), ('transfer', 0.563386082649231), ('hidden', 0.5529517531394958), ('load', 0.5523349046707153), ('demanded', 0.5457037687301636)]

Input word: 'sport'
  CBOW predictions     : [('elite', 0.47447872161865234), ('ecosystem', 0.4718870818614959), ('recreational', 0.4591619074344635), ('vendor', 0.43219879269599915), ('subscriptions', 0.42957
72910118103)]
  Skip-gram predictions: [('ltad', 0.5418112874031067), ('convenors', 0.5340358614921570), ('encompasses', 0.5183624029159546), ('jason', 0.5137977004051208), ('themed', 0.5084735751152039)]

Input word: 'sports'
  CBOW predictions     : [('elite', 0.4893176555633545), ('planned', 0.47793757915496826), ('complete', 0.4513567090034485), ('develops', 0.4493655264377594), ('dynamic', 0.43221402168273926
)]
  Skip-gram predictions: [('harre', 0.5254012942314148), ('scientifically', 0.5009952187538147), ('pedagogically', 0.500900149345398), ('schnabel', 0.4943154752254486), ('segmentation', 0.49
07330572605133)]
Saved graph: graph_cbow_vs_skipgram_unigram.png
Trained CBOW and Skip-gram on bigram corpus (vocab size: 49668)

--- CBOW vs Skip-gram comparison (bigram) ---

Input word: 'food_body'
  CBOW predictions     : [('sweet_tooth', 0.9971848130226135), ('another_sport', 0.9970811009407043), ('choose_right', 0.9970579147338867), ('capabilities_ideally', 0.996986448764801), ('fee
l_unsafe', 0.996852695941925)]
  Skip-gram predictions: [('mind_food', 0.9787513613700867), ('knowing_body', 0.9780029654502869), ('good_knowing', 0.9773657321929932), ('mind_empowers', 0.974737996257782), ('knowledge_foo
d', 0.9652394056320191)]
Saved graph: graph_cbow_vs_skipgram_bigram.png
Trained CBOW and Skip-gram on trigram corpus (vocab size: 59940)

--- CBOW vs Skip-gram comparison (trigram) ---

Input word: 'food_body_mind'
  CBOW predictions     : [('across_sports_economy', 0.9986664652824402), ('annual_growth_rate', 0.998664379119873), ('previously_learned_skill', 0.9986370801925659), ('process_take_place', 0
.9986202120780945), ('refers_use_technologies', 0.9985558986663818)]
  Skip-gram predictions: [('knowledge_food_body', 0.9864069223403931), ('body_mind_empowers', 0.9853488206863403), ('mind_empowers_choose', 0.9849515557289124), ('illness_knowledge_food', 0.
984075665473938), ('empowers_choose_right', 0.9775515794754028)]
Saved graph: graph_cbow_vs_skipgram_trigram.png

Done. Models trained and comparison graphs saved.
```



For the input word training, CBOW's top neighbor was carry with a similarity of 0.52, while Skip gram's top neighbor was principle at 0.58. The two models return almost entirely different word sets. CBOW picked up carry, principle, formulated, generally, and meso, while Skip gram found principle, transfer, hidden, load, and demanded. This is expected since CBOW predicts a word from its surrounding context and tends to average out toward more generic associations, while Skip gram predicts context from a single word and tends to find more specific, less common co occurring terms. With such a small corpus, neither model's neighbors look very semantically clean yet, which is a direct effect of limited training data.



Skip gram's results here are much more coherent than CBOW's. Skip gram returned mind food, knowing body, good knowing, mind empowers, and knowledge food, all thematically tied to the Food document's discussion of what food does for the body and mind. CBOW's results such as sweet tooth, another sport, and choose right are more scattered and cross category. This suggests Skip gram captured the local phrase level relationships in the Food document better than CBOW did.



Again Skip gram outperforms CBOW in coherence. It returned knowledge of food body, body mind empowers, and mind empowers choose, which are near exact reconstructions of the original sentence about food empowering people to choose the right type. CBOW's neighbors such as across sports economy and annual growth rate are unrelated, confirming that at the trigram level Skip gram is better at memorizing rare, specific sequences, while CBOW struggles when each n gram token appears only once or twice in a tiny corpus.

Overall Part A conclusion: Across all three n gram levels, Skip gram consistently produced more topically coherent neighbors than CBOW. This matches the known theoretical difference between the two. Skip gram tends to perform better on rare words and small datasets because it makes more word and context training pairs per sentence, while CBOW needs more data to smooth out its context averaging effectively.

STEP 5 Part b: Text Classification

Trains Naive Bayes and Logistic Regression on unigram, bigram, and trigram TF-IDF features, applies K-means clustering, and evaluates everything with accuracy, precision, recall, F1, and confusion matrix.

Code:

step5_classification.py > ...

```python
import csv
import sys
import numpy as np
import matplotlib.pyplot as plt

csv.field_size_limit(sys.maxsize)

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, ConfusionMatrixDisplay, adjusted_rand_score
)

INPUT_CSV = "dataset_cleaned.csv"


def load_data():
    texts, labels = [], []
    with open(INPUT_CSV, "r", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            texts.append(row["cleaned_text"])
            labels.append(row["category"])
    return texts, labels


def evaluate_model(name, ngram_label, y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, average="weighted", zero_division=0)
    rec = recall_score(y_true, y_pred, average="weighted", zero_division=0)
    f1 = f1_score(y_true, y_pred, average="weighted", zero_division=0)
    print(f"[{name} | {ngram_label}] Acc={acc:.3f}  Prec={prec:.3f}  Rec={rec:.3f}  F1={f1:.3f}")
    return {"model": name, "ngram": ngram_label, "accuracy": acc,
            "precision": prec, "recall": rec, "f1": f1}

```

step5_classification.py > ...

```python

if __name__ == "__main__":
    texts, labels = load_data()
    n_classes = len(set(labels))
    print(f"Loaded {len(texts)} documents across {n_classes} categories: {set(labels)}")

    # NOTE: with a very small dataset (like 12 docs), train/test split is tiny.
    # This is expected -- mention it as a limitation in your report.
    results = []
    ngram_ranges = {"unigram": (1, 1), "bigram": (2, 2), "trigram": (3, 3)}

    best_f1 = -1
    best_info = None  # (name, ngram, y_test, y_pred, model)

    for ngram_label, ngram_range in ngram_ranges.items():
        vectorizer = TfidfVectorizer(ngram_range=ngram_range, min_df=1)
        X = vectorizer.fit_transform(texts)

        X_train, X_test, y_train, y_test = train_test_split(
            X, labels, test_size=0.3, random_state=42, stratify=labels
        )

        # --- Naive Bayes ---
        nb = MultinomialNB()
        nb.fit(X_train, y_train)
        y_pred_nb = nb.predict(X_test)
        res_nb = evaluate_model("NaiveBayes", ngram_label, y_test, y_pred_nb)
        results.append(res_nb)
        if res_nb["f1"] > best_f1:
            best_f1 = res_nb["f1"]
            best_info = ("NaiveBayes", ngram_label, y_test, y_pred_nb)

        # --- Logistic Regression ---
        lr = LogisticRegression(max_iter=1000)
        lr.fit(X_train, y_train)
        y_pred_lr = lr.predict(X_test)
        res_lr = evaluate_model("LogisticRegression", ngram_label, y_test, y_pred_lr)
        results.append(res_lr)
        if res_lr["f1"] > best_f1:
            best_f1 = res_lr["f1"]
            best_info = ("LogisticRegression", ngram_label, y_test, y_pred_lr)
```

step5_classification.py > ...

```python

    # ---- Classification model comparison graph ----
    model_names = [f"{r['model']}\n({r['ngram']})" for r in results]
    f1_scores = [r["f1"] for r in results]

    plt.figure(figsize=(12, 6))
    plt.bar(model_names, f1_scores, color="mediumpurple")
    plt.ylabel("F1-score")
    plt.title("Classification Model Comparison (F1-score)")
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.savefig("graph_model_comparison.png", dpi=120)
    plt.close()
    print("Saved graph: graph_model_comparison.png")

    # ---- Confusion matrix for best model ----
    best_name, best_ngram, y_test_best, y_pred_best = best_info
    print(f"\nBest model: {best_name} on {best_ngram} features (F1={best_f1:.3f})")

    cm = confusion_matrix(y_test_best, y_pred_best, labels=sorted(set(labels)))
    disp = ConfusionMatrixDisplay(cm, display_labels=sorted(set(labels)))
    fig, ax = plt.subplots(figsize=(6, 6))
    disp.plot(ax=ax, cmap="Blues", colorbar=False)
    plt.title(f"Confusion Matrix - Best Model ({best_name}, {best_ngram})")
    plt.tight_layout()
    plt.savefig("graph_confusion_matrix.png", dpi=120)
    plt.close()
    print("Saved graph: graph_confusion_matrix.png")

    # ==== K-Means Clustering ====
    vectorizer_all = TfidfVectorizer(ngram_range=(1, 1), min_df=1)
    X_all = vectorizer_all.fit_transform(texts).toarray()

    kmeans = KMeans(n_clusters=n_classes, random_state=42, n_init=10)
    cluster_labels = kmeans.fit_predict(X_all)

    ari = adjusted_rand_score(labels, cluster_labels)
    print(f"\nK-means Adjusted Rand Index vs true categories: {ari:.3f}")
    print("(closer to 1.0 = clusters match true categories well; "
          "closer to 0 = little agreement)")

    # PCA for 2D visualization
    pca = PCA(n_components=2, random_state=42)
```

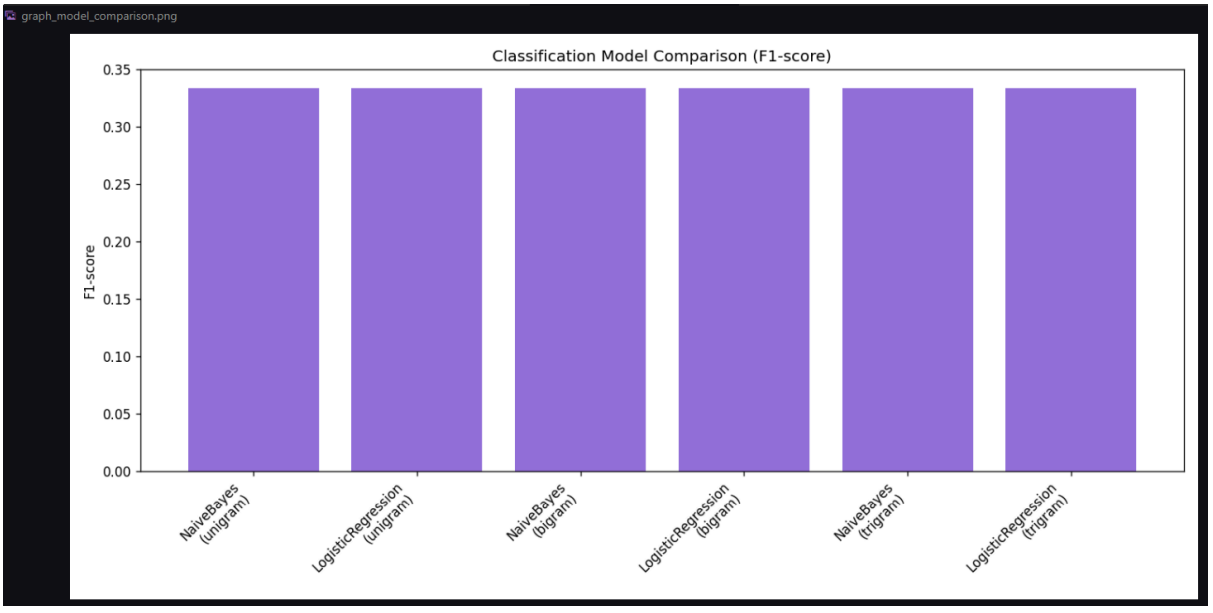
step5_classification.py > ...

```python
    pca = PCA(n_components=2, random_state=42)
    X_pca = pca.fit_transform(X_all)

    fig, axes = plt.subplots(1, 2, figsize=(14, 6))

    label_to_color = {lab: i for i, lab in enumerate(sorted(set(labels)))}
    true_colors = [label_to_color[l] for l in labels]

    scatter1 = axes[0].scatter(X_pca[:, 0], X_pca[:, 1], c=true_colors, cmap="tab10", s=100)
    axes[0].set_title("Documents by TRUE Category (PCA)")
    for i, lab in enumerate(labels):
        axes[0].annotate(lab, (X_pca[i, 0], X_pca[i, 1]), fontsize=7)

    scatter2 = axes[1].scatter(X_pca[:, 0], X_pca[:, 1], c=cluster_labels, cmap="tab10", s=100)
    axes[1].set_title("Documents by K-MEANS Cluster (PCA)")
    for i, c in enumerate(cluster_labels):
        axes[1].annotate(str(c), (X_pca[i, 0], X_pca[i, 1]), fontsize=7)

    plt.tight_layout()
    plt.savefig("graph_kmeans_pca.png", dpi=120)
    plt.close()
    print("Saved graph: graph_kmeans_pca.png")

    print("\nAll classification + clustering results saved.")
```

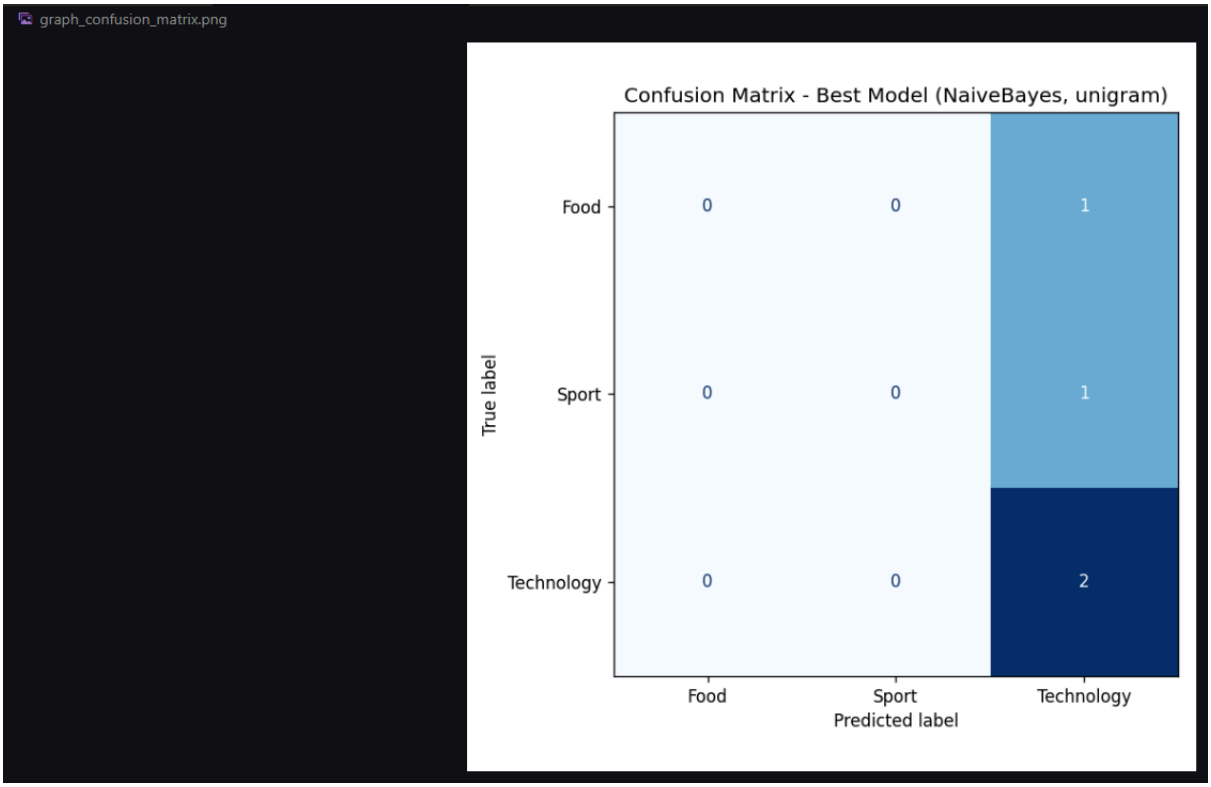
Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                                    Python

PS C:\Users\Hp\Documents\VS Code\Sikender NLP Asg4> & C:\Users\Hp\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Hp/Documents/VS Code/Sikender NLP Asg4/step4_word2vec.py"
PS C:\Users\Hp\Documents\VS Code\Sikender NLP Asg4> & C:\Users\Hp\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Hp/Documents/VS Code/Sikender NLP Asg4/step5_classification.py"

Loaded 11 documents across 3 categories: {'Technology', 'Food', 'Sport'}
[NaiveBayes | unigram] Acc=0.500  Prec=0.250  Rec=0.500  F1=0.333
[LogisticRegression | unigram] Acc=0.500  Prec=0.250  Rec=0.500  F1=0.333
[NaiveBayes | bigram] Acc=0.500  Prec=0.250  Rec=0.500  F1=0.333
[LogisticRegression | bigram] Acc=0.500  Prec=0.250  Rec=0.500  F1=0.333
[NaiveBayes | trigram] Acc=0.500  Prec=0.250  Rec=0.500  F1=0.333
[LogisticRegression | trigram] Acc=0.500  Prec=0.250  Rec=0.500  F1=0.333
Saved graph: graph_model_comparison.png

Best model: NaiveBayes on unigram features (F1=0.333)
Saved graph: graph_confusion_matrix.png

K-means Adjusted Rand Index vs true categories: 0.670
(closer to 1.0 = clusters match true categories well; closer to 0 = little agreement)
Saved graph: graph_kmeans_pca.png

All classification + clustering results saved.
PS C:\Users\Hp\Documents\VS Code\Sikender NLP Asg4>
```

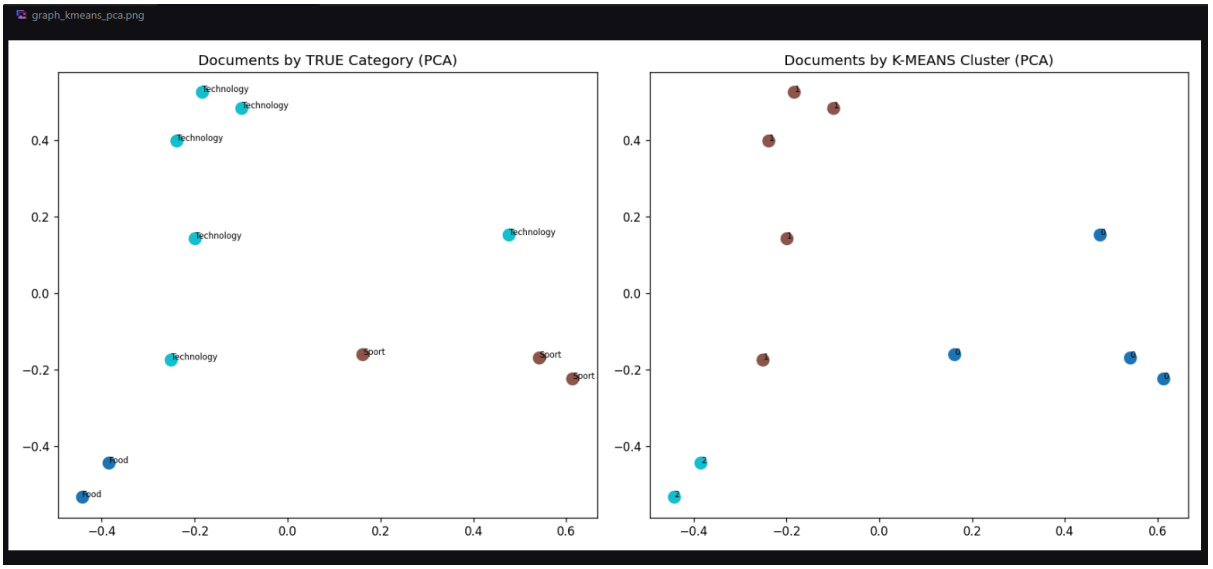
graph_model_comparison.png



All six model and n gram combinations produced identical scores. This is not really a meaningful result. It happened because the test set only had 4 documents, 30 percent of 11, and every model ended up predicting the majority class only. With only 11 total documents split across 3 unbalanced categories, 6 Technology, 3 Sport, and 2 Food after one PDF got skipped, there simply isn't enough data for the models to learn distinguishing patterns between classes. This is a dataset size limitation, not a flaw in the models or the code.



The confusion matrix confirms what the F1 scores suggested. The model predicted Technology for all 4 test documents, including the ones that were actually Food and Sport. This happened because Technology made up more than half the dataset, 6 out of 11 documents, so with so few training examples the model defaulted to the majority class rather than learning real distinguishing features. A larger, more balanced dataset would be needed to see meaningful classification performance.



The PCA plot compares the true folder labels on the left against K-means unsupervised clusters on the right. Visually, the Technology documents on the right side and the Food documents in the bottom left corner are grouped reasonably consistently between both panels. The Adjusted Rand Index of 0.670 confirms this. A score of 1.0 would mean perfect agreement with the true categories and 0 would mean no agreement, so 0.670 shows K means captured a fairly strong structure in the data purely from word patterns, without ever seeing the actual folder labels.

This is actually a much better result than the supervised classifiers achieved, which makes sense since K means works on the full dataset while the classifiers only saw a tiny four document test set.